```
"""
STEP 2: Install required libraries for NER implementation.

spaCy         → Traditional NLP pipeline
transformers  → Hugging Face transformer models
torch         → Backend deep learning framework
pandas        → Data handling and tabular storage
datasets      → To load standard NER datasets (optional)
"""

!pip install spacy transformers torch pandas datasets
!python -m spacy download en_core_web_sm
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.24.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (4.67.3)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (2
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy) (75.2.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (26.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.24.2)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1.4
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.2
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->trans
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1,<
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1,<3
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spa
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spa
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.
Requirement already satisfied: bliss<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spacy)
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim<1.0.0,>=0.3.0->spacy)
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.
```

```
"""
STEP 3: Import all required libraries.

Each library serves a specific purpose in NER implementation.
"""

import spacy  # Industrial NLP library (tokenization, tagging, NER)
from spacy import displacy  # Used to visualize named entities
```

```
from transformers import (
    pipeline,  # High-level API for inference
    AutoTokenizer,  # Loads tokenizer of pretrained model
    AutoModelForTokenClassification  # Loads NER model
)

import torch  # Deep learning backend
import pandas as pd  # For storing results in DataFrame
```

```
"""
STEP 4: Load spaCy pretrained English model.

The model contains:
- Tokenizer
- POS tagger
- Dependency parser
- Named Entity Recognizer (NER)
"""

# Load small English model
nlp = spacy.load("en_core_web_sm")

print("spaCy pipeline components:", nlp.pipe_names)
```

```
spaCy pipeline components: ['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer', 'ner']
```

```
"""
STEP 5: Create sample real-world sentences.

These sentences include:
- Political news
- Technology updates
- Sports events
- Business news
- International events
"""

sentences = [
    "Prime Minister Narendra Modi visited the United States in 2023.",
    "Apple Inc. launched the new iPhone 15 in California.",
    "Lionel Messi scored a goal for Inter Miami in the MLS final.",
    "Microsoft acquired OpenAI for billions of dollars.",
    "The Olympics 2024 will be held in Paris."
]

# Print sample sentences
for s in sentences:
    print(s)
```

```
Prime Minister Narendra Modi visited the United States in 2023.
Apple Inc. launched the new iPhone 15 in California.
Lionel Messi scored a goal for Inter Miami in the MLS final.
Microsoft acquired OpenAI for billions of dollars.
The Olympics 2024 will be held in Paris.
```

```
"""
STEP 6: Apply spaCy NER on each sentence.

For every sentence:
- Process text using spaCy pipeline
- Extract entity text
- Extract entity label
"""

print("===== spaCy NER Results =====\n")

for sentence in sentences:
    # Process sentence through spaCy pipeline
    doc = nlp(sentence)

    print(f"Sentence: {sentence}")

    # Loop through detected entities
    for ent in doc.ents:
        print(f"Entity: {ent.text} | Label: {ent.label_}")
```

```
    print("-" * 60)
```

```
===== spaCy NER Results =====

Sentence: Prime Minister Narendra Modi visited the United States in 2023.
Entity: Narendra Modi | Label: PERSON
Entity: the United States | Label: GPE
Entity: 2023 | Label: DATE
------------------------------------------------------------
Sentence: Apple Inc. launched the new iPhone 15 in California.
Entity: Apple Inc. | Label: ORG
Entity: 15 | Label: CARDINAL
Entity: California | Label: GPE
------------------------------------------------------------
Sentence: Lionel Messi scored a goal for Inter Miami in the MLS final.
Entity: Lionel Messi | Label: PERSON
Entity: MLS | Label: ORG
------------------------------------------------------------
Sentence: Microsoft acquired OpenAI for billions of dollars.
Entity: Microsoft | Label: ORG
Entity: OpenAI | Label: ORG
Entity: billions of dollars | Label: MONEY
------------------------------------------------------------
Sentence: The Olympics 2024 will be held in Paris.
Entity: Paris | Label: GPE
------------------------------------------------------------
```

```
"""
STEP 7: Store spaCy detected entities in a structured table.

Columns:
- Sentence
- Entity
- Label
"""

spacy_results = []  # Empty list to store results

for sentence in sentences:
    doc = nlp(sentence)

    for ent in doc.ents:
        # Append entity details
        spacy_results.append({
            "Sentence": sentence,
            "Entity": ent.text,
            "Label": ent.label_
        })

# Create DataFrame
spacy_df = pd.DataFrame(spacy_results)

# Display table
spacy_df
```

	Sentence	Entity	Label
0	Prime Minister Narendra Modi visited the Unite...	Narendra Modi	PERSON
1	Prime Minister Narendra Modi visited the Unite...	the United States	GPE
2	Prime Minister Narendra Modi visited the Unite...	2023	DATE
3	Apple Inc. launched the new iPhone 15 in Calif...	Apple Inc.	ORG
4	Apple Inc. launched the new iPhone 15 in Calif...	15	CARDINAL
5	Apple Inc. launched the new iPhone 15 in Calif...	California	GPE
6	Lionel Messi scored a goal for Inter Miami in ...	Lionel Messi	PERSON
7	Lionel Messi scored a goal for Inter Miami in ...	MLS	ORG
8	Microsoft acquired OpenAI for billions of doll...	Microsoft	ORG
9	Microsoft acquired OpenAI for billions of doll...	OpenAI	ORG
10	Microsoft acquired OpenAI for billions of doll...	billions of dollars	MONEY
11	The Olympics 2024 will be held in Paris.	Paris	GPE

```
"""
STEP 8: Visualize entities using displacy.

This will highlight entities with different colors:
PERSON → Orange
ORG    → Blue
GPE    → Green
DATE   → Purple
"""

# Visualize first sentence
displacy.render(nlp(sentences[0]), style="ent", jupyter=True)
```

Prime Minister Narendra Modi **PERSON** visited the United States **GPE** in 2023 **DATE** .

```
"""
STEP 9: Load pretrained Transformer model for token classification (NER).

Model used:
BERT-large fine-tuned on CoNLL-2003 dataset.
"""

model_name = "dbmdz/bert-large-cased-finetuned-conll03-english"

# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Load model
model = AutoModelForTokenClassification.from_pretrained(model_name)

# Create NER pipeline
ner_pipeline = pipeline("ner", model=model, tokenizer=tokenizer)

print("Transformer NER model loaded successfully!")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

```
config.json: 100%                                                    998/998 [00:00<00:00, 20.7kB/s]

tokenizer_config.json: 100%                                          60.0/60.0 [00:00<00:00, 644B/s]

vocab.txt: 213k/? [00:00<00:00, 3.21MB/s]

model.safetensors: 100%                                              1.33G/1.33G [00:12<00:00, 244MB/s]

Loading weights: 100%                                                391/391 [00:01<00:00, 307.61it/s, Materializing param=classifier.weight]
BertForTokenClassification LOAD REPORT from: dbmdz/bert-large-cased-finetuned-conll03-english
Key                       | Status     |  |
--------------------------+------------+--+-
bert.pooler.dense.bias    | UNEXPECTED |  |
bert.pooler.dense.weight  | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Transformer NER model loaded successfully!
```

```
"""
STEP 10: Apply Hugging Face Transformer NER.

For each sentence:
- Pass into pipeline
- Extract token
- Extract entity label
- Extract confidence score
"""

print("===== Hugging Face NER Results =====\n")

for sentence in sentences:

    results = ner_pipeline(sentence)
```

```
    print(f"Sentence: {sentence}")

    for r in results:
        print(f"Word: {r['word']} | Label: {r['entity']} | Score: {round(r['score'], 3)}")

    print("-" * 60)
```

```
===== Hugging Face NER Results =====

Sentence: Prime Minister Narendra Modi visited the United States in 2023.
Word: Na | Label: I-PER | Score: 0.9990000128746033
Word: ##ren | Label: I-PER | Score: 0.9929999709129333
Word: ##dra | Label: I-PER | Score: 0.9990000128746033
Word: Mo | Label: I-PER | Score: 0.9990000128746033
Word: ##di | Label: I-PER | Score: 0.9869999885559082
Word: United | Label: I-LOC | Score: 1.0
Word: States | Label: I-LOC | Score: 1.0
------------------------------------------------------------
Sentence: Apple Inc. launched the new iPhone 15 in California.
Word: Apple | Label: I-ORG | Score: 0.9990000128746033
Word: Inc | Label: I-ORG | Score: 0.9990000128746033
Word: iPhone | Label: I-MISC | Score: 0.9950000047683716
Word: 15 | Label: I-MISC | Score: 0.9950000047683716
Word: California | Label: I-LOC | Score: 1.0
------------------------------------------------------------
Sentence: Lionel Messi scored a goal for Inter Miami in the MLS final.
Word: Lionel | Label: I-PER | Score: 0.9990000128746033
Word: Me | Label: I-PER | Score: 0.9990000128746033
Word: ##ssi | Label: I-PER | Score: 0.9980000257492065
Word: Inter | Label: I-ORG | Score: 0.9990000128746033
Word: Miami | Label: I-ORG | Score: 0.9649999737739563
Word: MLS | Label: I-ORG | Score: 0.9210000038146973
------------------------------------------------------------
Sentence: Microsoft acquired OpenAI for billions of dollars.
Word: Microsoft | Label: I-ORG | Score: 1.0
Word: Open | Label: I-ORG | Score: 0.9990000128746033
Word: ##A | Label: I-ORG | Score: 0.9980000257492065
Word: ##I | Label: I-ORG | Score: 0.9980000257492065
------------------------------------------------------------
Sentence: The Olympics 2024 will be held in Paris.
Word: Olympics | Label: I-MISC | Score: 0.9940000176429749
Word: 202 | Label: I-MISC | Score: 0.6299999952316284
Word: Paris | Label: I-LOC | Score: 0.9990000128746033
------------------------------------------------------------
```

```
"""
STEP 11: Clean Transformer output.

Problem:
Transformer splits words into subwords (##).

Solution:
- Remove '##'
- Store clean entity names
- Save into DataFrame
"""

hf_results = []

for sentence in sentences:
    results = ner_pipeline(sentence)

    for r in results:
        # Remove subword token markers
        clean_word = r["word"].replace("##", "")

        hf_results.append({
            "Sentence": sentence,
            "Entity": clean_word,
            "Label": r["entity"],
            "Confidence Score": round(r["score"], 3)
        })

# Create DataFrame
hf_df = pd.DataFrame(hf_results)

hf_df
```

	Sentence	Entity	Label	Confidence Score
0	Prime Minister Narendra Modi visited the Unite...	Na	I-PER	0.999
1	Prime Minister Narendra Modi visited the Unite...	ren	I-PER	0.993
2	Prime Minister Narendra Modi visited the Unite...	dra	I-PER	0.999
3	Prime Minister Narendra Modi visited the Unite...	Mo	I-PER	0.999
4	Prime Minister Narendra Modi visited the Unite...	di	I-PER	0.987
5	Prime Minister Narendra Modi visited the Unite...	United	I-LOC	1.000
6	Prime Minister Narendra Modi visited the Unite...	States	I-LOC	1.000
7	Apple Inc. launched the new iPhone 15 in Calif...	Apple	I-ORG	0.999
8	Apple Inc. launched the new iPhone 15 in Calif...	Inc	I-ORG	0.999
9	Apple Inc. launched the new iPhone 15 in Calif...	iPhone	I-MISC	0.995
10	Apple Inc. launched the new iPhone 15 in Calif...	15	I-MISC	0.995
11	Apple Inc. launched the new iPhone 15 in Calif...	California	I-LOC	1.000
12	Lionel Messi scored a goal for Inter Miami in ...	Lionel	I-PER	0.999
13	Lionel Messi scored a goal for Inter Miami in ...	Me	I-PER	0.999
14	Lionel Messi scored a goal for Inter Miami in ...	ssi	I-PER	0.998
15	Lionel Messi scored a goal for Inter Miami in ...	Inter	I-ORG	0.999
16	Lionel Messi scored a goal for Inter Miami in ...	Miami	I-ORG	0.965
17	Lionel Messi scored a goal for Inter Miami in ...	MLS	I-ORG	0.921
18	Microsoft acquired OpenAI for billions of doll...	Microsoft	I-ORG	1.000
19	Microsoft acquired OpenAI for billions of doll...	Open	I-ORG	0.999
20	Microsoft acquired OpenAI for billions of doll...	A	I-ORG	0.998
21	Microsoft acquired OpenAI for billions of doll...	I	I-ORG	0.998
22	The Olympics 2024 will be held in Paris.	Olympics	I-MISC	0.994
23	The Olympics 2024 will be held in Paris.	202	I-MISC	0.630
24	The Olympics 2024 will be held in Paris.	Paris	I-LOC	0.999

```
"""
STEP 12: Create comparison table between spaCy and Transformer.
"""

comparison_data = {
    "Feature": [
        "Model Type",
        "Speed",
        "Accuracy (Qualitative)",
        "Context Handling",
        "Confidence Score",
        "Hardware Requirement"
    ],
    "spaCy": [
        "Statistical + CNN",
        "Very Fast",
        "Good",
        "Limited",
        "Not Provided",
        "CPU"
    ],
    "Hugging Face": [
        "Transformer (BERT)",
        "Slower",
        "Very High",
        "Excellent",
        "Provided",
        "GPU Recommended"
    ]
}

comparison_df = pd.DataFrame(comparison_data)
```

```
comparison_df
```

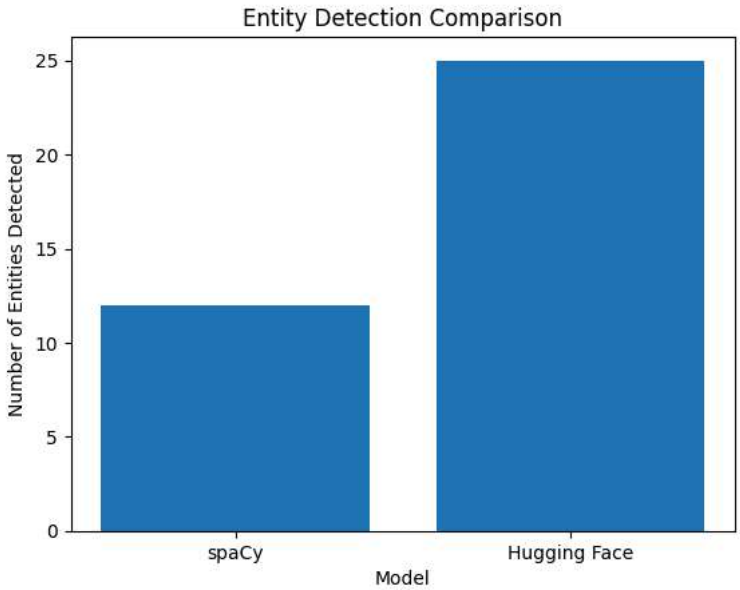
	Feature	spaCy	Hugging Face
0	Model Type	Statistical + CNN	Transformer (BERT)
1	Speed	Very Fast	Slower
2	Accuracy (Qualitative)	Good	Very High
3	Context Handling	Limited	Excellent
4	Confidence Score	Not Provided	Provided
5	Hardware Requirement	CPU	GPU Recommended

```
"""
Optional: Evaluate using seqeval library for F1-score.
"""

!pip install seqeval

from seqeval.metrics import classification_report, f1_score

# Example dummy evaluation (replace with real labels if dataset used)
true_labels = [["B-PER", "O", "B-LOC"]]
pred_labels = [["B-PER", "O", "B-LOC"]]

print("F1 Score:", f1_score(true_labels, pred_labels))
print(classification_report(true_labels, pred_labels))
```

```
Collecting seqeval
  Downloading seqeval-1.2.2.tar.gz (43 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43.6/43.6 kB 2.2 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.12/dist-packages (from seqeval) (2.0.2)
Requirement already satisfied: scikit-learn>=0.21.3 in /usr/local/lib/python3.12/dist-packages (from seqeval) (1.6.1)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.21.3->seqeval) (1.1
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.21.3->seqeval) (1.
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.21.3->seqev
Building wheels for collected packages: seqeval
  Building wheel for seqeval (setup.py) ... done
  Created wheel for seqeval: filename=seqeval-1.2.2-py3-none-any.whl size=16162 sha256=ca24941670f7d5cf9ab8a4fd6881b445765727e01
  Stored in directory: /root/.cache/pip/wheels/5f/b8/73/0b2c1a76b701a677653dd79ece07cfabd7457989dbfbdcd8d7
Successfully built seqeval
Installing collected packages: seqeval
Successfully installed seqeval-1.2.2
F1 Score: 1.0
              precision    recall  f1-score   support

         LOC       1.00      1.00      1.00         1
         PER       1.00      1.00      1.00         1

   micro avg       1.00      1.00      1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2
```

```
"""
STEP 14: Manual Error Analysis Template.

Students should inspect incorrect predictions manually.
"""

# Example incorrect case
sentence = "Microsoft acquired OpenAI for billions of dollars."
print("Sentence:", sentence)

print("\nspaCy Output:")
print([(ent.text, ent.label_) for ent in nlp(sentence).ents])

print("\nTransformer Output:")
print(ner_pipeline(sentence))
```

```
Sentence: Microsoft acquired OpenAI for billions of dollars.

spaCy Output:
```

```
[('Microsoft', 'ORG'), ('OpenAI', 'ORG'), ('billions of dollars', 'MONEY')]

Transformer Output:
[{'entity': 'I-ORG', 'score': np.float32(0.99959713), 'index': 1, 'word': 'Microsoft', 'start': 0, 'end': 9}, {'entity': 'I-ORG'
```

```
"""
Graph 1: Compare total number of entities detected
by spaCy and Hugging Face Transformer.
"""

import matplotlib.pyplot as plt

# Count number of detected entities
spacy_count = len(spacy_df)
hf_count = len(hf_df)

# Create bar graph
plt.figure()
plt.bar(["spaCy", "Hugging Face"], [spacy_count, hf_count])

plt.xlabel("Model")
plt.ylabel("Number of Entities Detected")
plt.title("Entity Detection Comparison")

plt.show()
```